PLASTICTRACE: BLOCKCHAIN-BASED PLASTIC WASTE TRACEABILITY SYSTEM

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Use Case Report

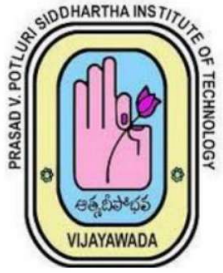
submitted by

GADIRAJU SIVA SAI NARAYANA

23501A0539

Under the guidance of

Mr. A. Prashant, Asst. Prof.



Department of Computer Science and Engineering

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

2026-27

Prasad V Potluri Siddhartha Institute of Technology

(Permanently affiliated to JNTU-Kakinada, Approved by AICTE)

(An NBA & NAAC accredited and ISO 9001:2015 certified institute)

Kanuru, Vijayawada-520 007

CERTIFICATE

This is to certify that the Use Case report entitled "PlasticTrace: Blockchain-Based Plastic Waste Traceability System" is being submitted by Gadiraju Siva Sai Narayana (23501A0539), as part of Assignment-2 for the Blockchain Technology (23CS4701B) course in B.Tech 4-1 during the academic year 2026-27.

Course Coordinator
Mr. A. Prashant
Assistant Professor,
Department of CSE,
PVPSIT, Vijayawada

Head of the Department
Dr. A. Jayalakshmi,
Professor and Head,
Department of CSE,
PVPSIT, Vijayawada

MARKS
Assignment-2 : ---/5

INDEX

S. No.	Chapter	Page No.
1	Introduction	04
2	Background	05
3	Blockchain Basics	07
4	Use Case Overview	08
5	Implementation	11
6	Benefits	15
7	Challenges	16
8	Conclusion	17
9	SDGs Addressed	18
10	References	19
11	Appendix A	20

1. INTRODUCTION

1.1. Overview of Plastic Waste Traceability

Plastic waste recycling is the process of collecting used plastic, sorting it by resin type, reprocessing it into secondary raw material and supplying that material to manufacturers. Plastic waste passes through several loosely coordinated stakeholders, including waste collectors, collection or sorting centers, recyclers and product manufacturers. A batch identifier links the plastic type, quantity, location, current custodian and the history of every handover, so that the journey of a lot can be reviewed from collection to its final use.

1.2. Relevance to Plastic Recycling Supply Chains

Recycling workflows need accurate records of how much plastic was collected, who currently holds it, where it was moved and how much usable material resulted from reprocessing. When each participant keeps these details in private spreadsheets or proprietary systems, the records are difficult to compare across the chain, and claims about recycled content are hard to verify independently. PlasticTrace addresses this by recording the chain of custody of each plastic batch on an Ethereum Virtual Machine (EVM) compatible blockchain, and by presenting the result through a React-based web interface.

1.3. Problem Statement

The project addresses four problems found in conventional recycling workflows. First, fragmented data silos make cross-supply-chain tracing difficult. Second, shipped quantities frequently differ from received quantities because of moisture loss, sorting loss or inaccurate scales, which leads to untracked loss disputes. Third, centrally maintained records can be altered retroactively to inflate recycled-content quotas or to pass environmental audits. Fourth, consumers and ESG auditors cannot independently verify whether a product actually contains certified post-consumer recycled (PCR) material, which allows greenwashing to persist.

1.4. Objectives

The objectives of the project are to implement a decentralized chain of custody in which custody changes hands only through a two-phase handshake of dispatch followed by receipt confirmation; to enforce role-based permissions in the smart contract so that only authorized parties can register, transfer or reprocess a batch; to validate on-chain that recycled output plus residual waste equals the total mass processed; to provide a transparent audit trail through a blockchain explorer and batch timeline; and to operate entirely on a local Hardhat blockchain so that no testnet faucets, paid gas or external databases are required.

The system therefore keeps all lifecycle state in the PlasticWasteTraceability smart contract and reads it directly from the React front end through Ethers.js. Every state change is a signed transaction, and every lifecycle step is stored as an append-only record. The reliability of the recorded information still depends on honest data entry at the physical boundary, which is discussed in the chapter on challenges.

2. BACKGROUND

2.1. Traditional Plastic Recycling Chains

Traditional recycling chains depend on paper waybills, verbal confirmation, email and spreadsheets maintained separately by each stakeholder. A transfer may be recorded by the sender but not by the receiver, weights may be estimated rather than measured consistently, and no participant has a single view of the whole chain. Centralized databases and enterprise software improve access to information within one organization, but they do not give several organizations with different commercial interests a shared record that none of them controls.

2.2. Transparency and Auditability

Transparency means that authorized users and auditors can inspect when a batch was registered, who held it, how much of it moved and what was produced from it. Auditability means that this history cannot be quietly rewritten. PlasticTrace stores every lifecycle event, with the acting address, recipient, quantity, timestamp, location and notes, in an append-only array inside the smart contract, and exposes it through a timeline view and a blockchain explorer.

2.3. Existing System Limitations

A conventional database supports fast searches and updates, but whoever administers it can alter or delete entries, and an external auditor must trust the administrator. Mass-balance reports are usually self-reported spreadsheets, and certificates are paper documents that the buyer must trust the issuer to have produced honestly. A blockchain makes committed changes detectable, although it cannot prove that a weighbridge reading or a laboratory result was correct. Sensors, receiving checks and periodic audits therefore remain essential.

2.4. Proposed System

The proposed system is an end-to-end decentralized application consisting of a Solidity smart contract, a local Hardhat blockchain and a React dashboard. Each plastic lot is given a unique Batch ID, role-based access control limits who may act, and the contract enforces a mass-balance conservation equation at the recycling stage. The comparison below summarizes how the proposed system differs from conventional practice.

Table 2.1: Conventional recycling systems compared with PlasticTrace

Dimension	Conventional Recycling Systems	PlasticTrace Blockchain System
Data storage	Centralized SQL/NoSQL databases or Excel sheets	Decentralized EVM ledger
Custody handover	Paper waybills, verbal confirmation, email	Smart-contract two-phase handshake with event logs
Data integrity	Vulnerable to admin alteration, deletion or backdating	Append-only, cryptographically hashed, immutable
Mass-balance audit	Manual self-reported spreadsheets	Programmatically enforced in Solidity bytecode
Public verification	Trust in third-party paper certificates	Direct query of contract state via Batch ID
Single point of failure	Central server / IT department	Distributed network of validating nodes

2.5. Real-World Precedents

The project is grounded in current industrial practice. IBM and Plastic Bank deployed private blockchain infrastructure on IBM LinuxONE to track ocean-bound plastic in developing regions and to reward local collectors with digital tokens redeemable for basic necessities. In Japan, Asahi Kasei took part in an industrial consortium trial that used digital product passports and blockchain to certify recycled plastic content across automotive and packaging supply chains. Both examples show that traceability of recycled material is an active industrial concern, while PlasticTrace demonstrates the same idea as an academic prototype.

3. BLOCKCHAIN BASICS

3.1. Distributed Ledger

A blockchain is an ordered ledger of transactions maintained by a network of nodes, where each block is cryptographically linked to the one before it. PlasticTrace targets any EVM-compatible network. For development and demonstration it runs on a local Hardhat node at http://127.0.0.1:8545, which provides twenty funded test accounts and requires no external network, tokens or fees.

3.2. Batch Record Integrity

The contract defines Batch and LifecycleRecord structures. A Batch holds its identifier, plastic type, initial and current quantity, recycled output, residual waste, collection location, original collector, current custodian, pending recipient, status, registration and last-updated timestamps, and an existence flag. A LifecycleRecord stores the event type, acting address, recipient, quantity, timestamp, location, notes and details.

3.3. Smart Contracts

PlasticWasteTraceability.sol is written in Solidity and provides functions to register a batch, initiate a transfer, confirm receipt, record reprocessing, read a batch and its history, and compute system-wide metrics. An administrator account deploys the contract and assigns roles. Each protected function checks the caller's role, the batch status and the supplied values, and reverts with a descriptive message when a rule is violated.

3.4. Events and Traceability

Every state-changing function appends a LifecycleRecord and emits an event, such as BatchRegistered when a batch is created and StakeholderRoleUpdated when the administrator assigns a role. The lifecycle event types are Registration, TransferInitiated, ReceiptConfirmed, ProcessingRecorded and Completed. Transaction hashes and block numbers returned by the network allow a user to relate each step in the timeline to a specific block.

3.5. On-chain and Off-chain Data

In this prototype all lifecycle state is held on-chain, so the front end can read it directly from the contract without a separate application database. In a production setting, bulky or sensitive material such as laboratory quality certificates, moisture test reports and photographs of bales should be stored off-chain, for example on IPFS, with only their hashes recorded on-chain. Sensitive commercial information should not be published on a shared ledger without clear justification and privacy controls.

3.6. Why Blockchain is Appropriate

A blockchain suits this problem for four reasons. Collectors, municipal hubs, recyclers and brand owners often have conflicting commercial incentives, so no single party's server is trusted by all. Once a block is mined, its timestamps and actor signatures cannot be altered to satisfy a later audit demand. Each lifecycle claim is signed with a private key, which provides non-repudiation during disputes. Finally, the technology guarantees data and record integrity but not physical reality; sensors, weighbridge calibration and laboratory resin tests are still needed to verify the material itself.

4. USE CASE OVERVIEW

4.1. Stakeholders

The system supports five on-chain roles, which are Admin, Collector, CollectionCenter, Recycler and Manufacturer, and treats any other address as having no role. The administrator deploys the contract and assigns roles. Collectors register raw plastic batches, collection centers sort and bale them, recyclers reprocess them and manufacturers receive the secondary material. Members of the public have no write access but can verify a batch through the public verification page.

Table 4.1: Stakeholder roles

Role	Persona in demonstration	Responsibility
Admin	System Admin / Auditor	Deploys the contract and assigns stakeholder roles
Collector	GreenCollect Services	Aggregates and registers raw plastic waste
CollectionCenter	City EcoSorting Hub	Sorts, bales and transfers plastic
Recycler	PureCycle Polymers	Mechanical reprocessing and mass-balance recording
Manufacturer	EcoPackaging Ltd	Produces secondary goods and packaging
None	Public / Unauthorized	Read-only verification; writes are rejected on-chain

4.2. Functional Scope

The repository includes batch registration, two-phase custody transfer and receipt, mass-balance reprocessing, batch tracking with a chronological timeline, a blockchain explorer for blocks and events, a public authenticity verification portal, a dashboard with live on-chain key performance indicators, a persona switcher for demonstration and optional MetaMask signing. An About page summarizes the project, its SDG alignment and its references.

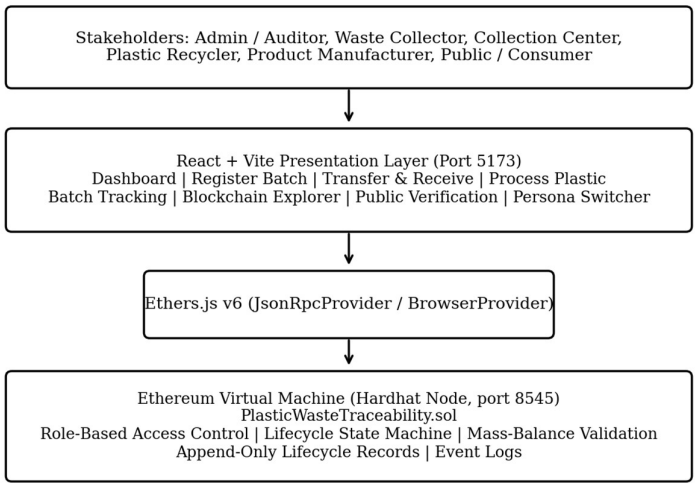
4.3. System Architecture

The system has a presentation layer, a Web3 client layer and a blockchain layer. The React and Vite front end provides the user interface. Ethers.js version 6 connects it to the Hardhat node through a JSON-RPC provider or a browser signer, and the PlasticWasteTraceability contract executes on the Ethereum Virtual Machine. There is no separate backend server or external database.

Table 4.2: Architecture layers

Layer	Main technologies / responsibilities
Presentation	React 18, Vite 5, JavaScript, plain CSS and Lucide React icons; dashboard, registration, transfer, processing, tracking, explorer and verification pages
Web3 client	Ethers.js 6.13.5 with a JsonRpcProvider or BrowserProvider and a configured set of demonstration signers
Blockchain	Solidity ^0.8.24 (EVM target Paris), Hardhat 2.22.18, local node on port 8545 and the PlasticWasteTraceability contract

Figure 4.1: Logical architecture of the PlasticTrace system



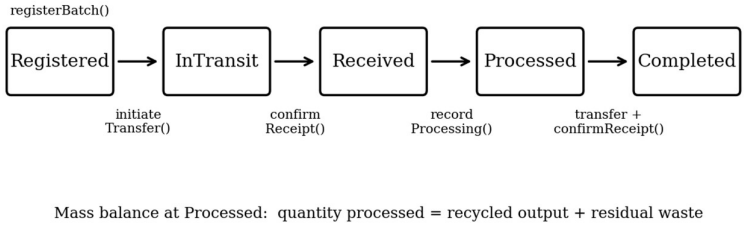
4.4. Architecture Components

Presentation layer: React 18 components for the header, sidebar, role switcher modal and transaction feedback banner, together with page components for each workflow. Web3 layer: a blockchain service module containing the Ethers.js client and the account configuration, and a contractData.json file holding the exported ABI and deployed contract address. Blockchain layer: the Solidity contract with its role-based access control, batch lifecycle state machine, mass-balance validation, append-only lifecycle records and events, compiled, tested and deployed with Hardhat.

4.5. Main Use Case Flow

A collector registers a batch with its Batch ID, resin type, quantity and collection location. The collector then initiates a transfer to a collection center, which places the batch in the InTransit state. The collection center confirms receipt and becomes the custodian, and the batch moves on to a recycler through the same two-phase handshake. The recycler records processing with the processed mass, the recycled output and the residual waste, and the contract checks that processed mass equals output plus residual. Finally the recycler transfers the recycled material to a manufacturer, and the batch is marked Completed once the manufacturer confirms receipt.

Figure 4.2: Batch lifecycle state flow



Each of these steps is a signed transaction. The interface shows the transaction hash and block number after confirmation, and a transaction should be treated as final only after the network reports success.

4.6. Tracking and Verification

The Batch Tracking page searches by Batch ID and shows a chronological timeline with each step, the acting address and the transaction reference. The Public Verification page presents a verification certificate for a batch by querying the contract directly. Verification confirms that the recorded history is intact and unaltered; it does not confirm that the physical plastic matches the record, which needs separate inspection.

5. IMPLEMENTATION

5.1. Development Environment

The repository is organized into blockchain and frontend directories with a documentation folder. The blockchain directory contains the Solidity contract, Hardhat configuration, deployment and seed scripts and the test suite. The frontend directory contains the React application built with Vite. Development requires Node.js (v18, v20 or v22 LTS), npm v9 or later and a modern web browser.

Table 5.1: Technology stack

Component	Technology
Smart contract language	Solidity ^0.8.24 (EVM target: Paris)
Local blockchain and testing	Hardhat 2.22.18 with @nomicfoundation/hardhat-toolbox; Mocha/Chai tests
Frontend framework	React 18 with Vite 5, JavaScript and plain CSS
Web3 client	Ethers.js 6.13.5 (JSON-RPC provider and browser signer)
Icons and UI	Lucide React; CSS variables for the sustainability theme (emerald, teal, navy)
Runtime	Node.js LTS and npm

5.2. Role-Based Access and Stakeholder Accounts

The contract defines the roles None, Admin, Collector, CollectionCenter, Recycler and Manufacturer. The deploying account becomes the administrator, and only the administrator can assign roles; role assignment by a non-admin or to the zero address is rejected. For demonstration, the standard deterministic Hardhat accounts are mapped to supply-chain personas: Account #0 is the System Admin, #1 the Collector, #2 the Collection Center, #3 the Recycler, #4 the Manufacturer and #5 an unauthorized public user. Account #5 is used to show that the contract rejects unauthorized actions.

5.3. Batch Data Model

The Batch structure records batchId, plasticType, initialQuantity, currentQuantity, recycledOutput, residualWaste, collectionLocation, collector, currentCustodian, pendingRecipient, status, registrationTimestamp, lastUpdatedTimestamp and an exists flag. The BatchStatus enumeration contains Registered, InTransit, Received, Processed and Completed. The LifecycleRecord structure stores eventType, actor, recipient, quantity, timestamp, location, notes and details, and every action appends one such record to the batch history.

5.4. Custody Transfer and Receipt Handshake

Custody changes through two separate transactions. The current custodian calls initiateTransfer, naming an authorized recipient, a quantity and a location, which sets the pending recipient and moves the batch to InTransit. The designated recipient then calls confirmReceipt, which updates the custodian and sets the status to Received, or to Completed when the recipient is a manufacturer. The contract rejects a transfer if the caller is not the current custodian, if the recipient is unauthorized or the role sequence is invalid, if the batch is

already in transit, or if the quantity exceeds the batch quantity. It also rejects a receipt confirmation from any account other than the pending recipient.

5.5. Mass-Balance Validation

At the recycling stage the authorized recycler calls recordProcessing with the processed quantity, the recycled output, the residual waste, a location and notes. The caller must be the current custodian, and the contract enforces the conservation equation that quantityProcessed equals recycledOutput plus residualWaste. Processing is also rejected if the quantity exceeds the mass available in the batch or if the caller is not a recycler. This check prevents a recycler from claiming more secondary material than the input could have produced. The front end shows a live mass-balance calculator so that the user can see whether the figures balance before submitting the transaction.

5.6. Smart Contract Functions

The contract exposes the following main functions. Each write function is guarded by role and state checks and appends a lifecycle record.

Table 5.2: Main contract functions

Function	Purpose and main validations
registerBatch()	Registers a batch; callable by an authorized Collector or the Admin; rejects empty or duplicate Batch IDs and zero quantity
initiateTransfer()	Starts a custody transfer; caller must be the current custodian; recipient must be an authorized stakeholder; status becomes InTransit
confirmReceipt()	Completes the handshake; caller must be the pending recipient; updates custodian and status
recordProcessing()	Records reprocessing by an authorized Recycler; enforces processed mass = recycled output + residual waste
getBatch() / getBatchHistory()	View functions returning batch details and the array of lifecycle events
getSystemMetrics()	Computes network-wide totals for batches, input mass, in-transit batches and recycled output

5.7. Frontend Modules

The React application is organized around a sidebar and eight pages. State and routing are managed in App.jsx, and the blockchain service module in the services directory creates the Ethers.js client and holds the account configuration.

Table 5.3: Frontend pages

Page	Purpose
Dashboard	Live on-chain KPIs, charts and recent batches
Register Batch	Form to register a batch on-chain
Transfer & Receive	Tabs for the dispatch and receipt-confirmation handshake
Process Plastic	Mass-balance recycling calculator and processing submission
Batch Tracking	Search by Batch ID and chronological lifecycle timeline
Blockchain Explorer	Blocks, events and raw logs
Public Verification	Consumer-facing verification certificate for a batch
About Project	Viva guide, SDG alignment and references

5.8. Blockchain Service Integration

The deployment script publishes the contract, assigns stakeholder roles on-chain and exports the ABI and contract address to frontend/src/contracts/contractData.json. The front end loads this file, so after any redeployment or reset of the local blockchain the exported configuration must be refreshed. Transactions can be signed either with the built-in persona switcher, which uses the Hardhat demonstration signers, or with MetaMask connected to a manually added network named Hardhat Localhost with RPC URL http://127.0.0.1:8545, chain ID 31337 and currency symbol ETH.

5.9. Running the System

Dependencies are installed with npm install in both the blockchain and frontend directories. The local blockchain is started with npx hardhat node, which leaves a node running on port 8545. In a second terminal, npm run deploy deploys the contract, assigns roles and exports the configuration, and the optional npm run seed command executes a complete PET batch walkthrough, PL-1001, and seeds additional batches. Finally, npm run dev in the frontend directory starts the application, which is served at http://localhost:5173.

5.10. Automated Testing

The project includes 23 automated Mocha/Chai tests, run with npx hardhat test, covering the business rules, access controls and mass-balance calculations. The tests are grouped into six areas, summarized below.

Table 5.4: Smart contract test coverage

Test group	Examples of behavior verified
Role management and access control	Admin set at deployment; admin-only role assignment; zero-address rejection
Batch registration	Successful registration with event; duplicate ID, unauthorized caller, empty ID and zero quantity rejected
Transfer and custody handover	Custodian-only transfers; invalid recipient, in-transit batch and excess quantity rejected
Receipt confirmation	Pending recipient confirms and becomes custodian; wrong recipient or wrong state rejected
Processing and mass balance	Valid mass balance accepted; violation, excess quantity and non-recycler rejected
History and system metrics	Immutable chronological audit trail; correct global metrics; listing of batch IDs

The repository documentation reports that all 23 tests pass. These tests exercise the contract on a local Hardhat network; they do not constitute performance or security validation for a production deployment.

5.11. Demonstration Scenario

A typical live demonstration follows ten steps. The presenter shows the dashboard KPIs, then registers batch PL-2001 as the Collector with 100 kg of PET at a collection center in Vijayawada, and shows the transaction hash and block number. Registering PL-2001 again demonstrates duplicate rejection, and switching to the unauthorized account demonstrates rejection of an unauthorized registration. The Collector then transfers the batch to the Collection Center, which confirms receipt, after which the batch is transferred to the Recycler and confirmed. The Recycler processes 100 kg into 80 kg of recycled output and 20 kg of residual waste, and the live calculator confirms conservation of mass before the transaction is mined. The presenter ends by searching PL-2001 in Batch Tracking to walk through the timeline and on the Public Verification page to show the verification certificate.

5.12. Important Implementation Notes

The ABI and contract address in the front end must match the contract deployed on the chosen network, and they must be updated whenever the local blockchain is reset. The demonstration signers use publicly known Hardhat keys and must never be used on a real network. The persona switcher is a demonstration aid and does not replace wallet-based authentication in a deployed system. This report describes the implementation as documented in the project repository; it does not claim a production deployment or integration with external ERP, sensor or weighbridge systems.

6. BENEFITS

6.1. Chain-of-Custody Integrity

The two-phase handshake means that custody moves only when the sender dispatches and the intended recipient confirms. A batch cannot be silently reassigned, and the contract rejects transfers made by anyone other than the current custodian. This removes the ambiguity that arises when a sender records a shipment that the receiver never acknowledges.

6.2. Traceability and Transparency

Every lifecycle step is stored as an append-only record with the actor, recipient, quantity, timestamp and location. Users can follow a batch from registration to completion through the timeline, and auditors can examine blocks, events and transaction hashes in the explorer. Anyone with access to the network can query a batch by its Batch ID.

6.3. Access Control and Accountability

Role-based permissions in the contract ensure that only authorized collectors register batches, only current custodians initiate transfers and only authorized recyclers record reprocessing. Because each action is signed by the caller's key, participants cannot later deny having performed it, which supports non-repudiation in disputes.

6.4. Mass-Balance Assurance

Enforcing the conservation equation in the contract makes it impossible to record a processing step whose recycled output and residual waste do not add up to the mass processed. This reduces the room for inflated secondary-material claims, although the input figures themselves must still be measured honestly.

6.5. Zero-Cost, Reproducible Environment

Because the system runs on a local Hardhat blockchain, it needs no testnet faucets, paid gas or external databases. The deployment and seed scripts and the 23 automated tests allow the demonstration to be reproduced reliably, which is valuable for academic evaluation.

6.6. Public Verification and Analytics

The Public Verification portal lets a consumer or ESG auditor check a batch directly against contract state instead of trusting a paper certificate. The dashboard summarizes total batches, collected input, batches in transit, recycled output and reprocessing yield from on-chain data. These summaries are only as complete as the data recorded.

The system can improve the visibility and reviewability of recorded recycling operations, but it cannot eliminate inaccurate weights, fraud at the point of data entry or physical losses. Sensor integration, receiving checks, staff procedures and laboratory testing remain necessary for reliable recycled-content claims.

7. CHALLENGES

7.1. Physical–Digital Gap

A blockchain records what authorized participants submit; it does not independently verify the weight of a bale, the resin type or whether a transfer physically took place. This is often called the oracle problem. In production, IoT-connected weighbridges, automated camera-based resin classifiers and RFID or barcode tagging would be needed to capture physical facts automatically.

7.2. Data Quality

Reliable weighing, receiving checks, moisture allowances and user accountability are needed so that recorded quantities reflect real operations. If a collector enters an incorrect quantity at registration, the contract will preserve that figure faithfully, and later mass-balance checks will balance against it.

7.3. Privacy and Key Management

Commercial information such as supplier names, prices and proprietary process details should not be published on a public or shared ledger without clear justification. The prototype uses deterministic Hardhat demonstration accounts whose keys are public, so a real deployment must use securely managed keys, keep environment files out of source control and restrict role assignment to a governed administrator.

7.4. Scalability and Cost

The local Hardhat environment is useful for development, but it does not demonstrate production scalability. On a public network, transactions introduce confirmation delays, fees and throughput limits. Performance should be evaluated with realistic batch volumes, concurrent users and network failure scenarios before any wider deployment.

7.5. Confidential Verification and Document Storage

Recyclers may want to prove compliance with mass-balance conservation without revealing commercial pricing or supplier identities. Zero-knowledge proofs such as ZK-SNARKs could allow this. Laboratory certificates, moisture test reports and bale photographs are too large to store on-chain and would be better held on decentralized storage such as IPFS, with their hashes recorded in the contract.

7.6. Future Improvements

Potential extensions include integration of IoT weighbridges and RFID or barcode tagging, zero-knowledge proofs for confidential compliance checks, IPFS storage for certificates and images, deployment to a public testnet with wallet-based authentication, and a governance process for role assignment. These are recommendations for future work, not claims that every feature is currently implemented.

8. CONCLUSION

8.1. Summary

PlasticTrace combines a Solidity smart contract, a local Hardhat blockchain and a React dashboard to record the lifecycle of plastic waste batches from collection to manufacturing. The repository includes role-based access control, a two-phase custody handshake, mass-balance validation, an append-only audit trail, a blockchain explorer, public verification and 23 automated tests. The combination is intended to make recycling claims easier to audit and harder to alter.

8.2. Outcome

The system provides a working foundation for registering plastic batches, transferring custody with explicit confirmation, recording reprocessing under a conservation rule and verifying the resulting history. The contract guarantees the integrity of recorded data, while the accuracy of that data depends on trustworthy input at the physical boundary and on appropriate testing and configuration.

8.3. Future Scope

Future work may include IoT and RFID integration for automatic capture of weights and identities, zero-knowledge proofs for confidential mass-balance verification, IPFS storage for quality certificates, deployment on a public testnet with wallet-based authentication and broader testing under realistic volumes and failure conditions.

Additional improvements could include multiple resin-specific yield analytics, integration with extended producer responsibility reporting and clearer exception workflows for lost, contaminated or disputed batches. These additions should be prioritized according to stakeholder feedback and validated through controlled testing.

9. SDGs ADDRESSED

SDG 12: Responsible Consumption and Production

SDG 12 is the primary focus of the project. Target 12.5 aims to substantially reduce waste generation through prevention, reduction, recycling and reuse. PlasticTrace provides verified tracking of secondary polymers and can help manufacturers legitimately demonstrate that they meet recycled-packaging mandates. This is a potential contribution; the project does not measure actual reductions in waste or resource consumption.

SDG 11: Sustainable Cities and Communities

Target 11.6 focuses on reducing the environmental impact of cities, including through sound municipal solid waste management. Transparent records of how collected plastic moves through sorting hubs and recyclers can support accountability in municipal waste handling. The effect depends on adoption by municipal and private operators.

SDG 13 and SDG 14: Indirect Benefits

Diverting plastic from landfills and waterways may indirectly help to reduce greenhouse gas emissions and marine plastic debris. These are indirect and possible benefits rather than outcomes measured by the current prototype.

10. REFERENCES

Project Source Code

PlasticTrace: Blockchain-Based Plastic Waste Traceability System. Source code repository, accessed 26 October 2026. https://github.com/Sai-Narayana/plastic_trace

Solidity Documentation

Solidity documentation. Smart contract language documentation. https://docs.soliditylang.org/

Hardhat Documentation

Hardhat documentation. Ethereum development environment and contract deployment. https://hardhat.org/docs

Ethers.js Documentation

Ethers.js v6 documentation. JavaScript library for interacting with Ethereum-compatible networks. https://docs.ethers.org/v6/

React Documentation

React documentation. React user-interface library. https://react.dev/

IBM and Plastic Bank

IBM case study: Plastic Bank systems on IBM LinuxONE. https://www.ibm.com/case-studies/plastic-bank-systems-linuxone

Asahi Kasei

Asahi Kasei news release, July 2022, on blockchain-based certification of recycled plastic. https://www.asahi-kasei.com/news/2022/e220706

11. APPENDIX A

A.1. Repository Structure

The repository contains a blockchain directory with the contracts folder (PlasticWasteTraceability.sol), scripts (deploy.js and seed.js), the test folder with the 23-test suite and hardhat.config.js. The frontend directory contains src with components (Header, Sidebar, RoleSwitcherModal, TransactionFeedback), pages (Dashboard, RegisterBatch, BatchTracking, TransferReceive, ProcessPlastic, BlockchainExplorer, PublicVerification, AboutProject), the services/blockchain.js client, contracts/contractData.json, styles and the App and main entry files. The docs directory holds architecture.md and diagrams.md, and the root contains a package.json with convenience scripts and the README.

A.2. Main Data Entities

The principal on-chain entities are Batch, LifecycleRecord and the Role, BatchStatus and EventType enumerations. Batch records include identifiers, plastic type, quantities, custodian and status. Lifecycle records capture the event type, actor, recipient, quantity, timestamp, location, notes and details.

A.3. Preconfigured Stakeholder Accounts

The deterministic Hardhat accounts used in the demonstration are listed below. These keys are public and are suitable only for local development.

Acct	Persona	Address	Role
#0	System Admin / Auditor	0xf39Fd6e51aad88F6F4ce6aB8827279cffFb92266	Admin
#1	GreenCollect Services	0x70997970C51812dc3A010C7d01b50e0d17dc79C8	Collector
#2	City EcoSorting Hub	0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC	CollectionCenter
#3	PureCycle Polymers	0x90F79bf6EB2c4f870365E785982E1f101E93b906	Recycler
#4	EcoPackaging Ltd	0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65	Manufacturer
#5	Public / Unauthorized	0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc	None

A.4. Configuration Checklist

Install dependencies in the blockchain and frontend directories, start the Hardhat node, deploy the contract, and confirm that contractData.json has been regenerated before starting the front end. After any reset of the local node, redeploy and refresh the exported configuration. If MetaMask is used, add the Hardhat Localhost network with chain ID 31337 and keep any imported keys limited to local test accounts.

A.5. Scope Note

This report describes components and workflows documented in the project repository. It does not claim that the system is deployed in production, that it has been validated beyond the automated contract tests, or that external IoT, ERP or weighbridge systems are integrated. Contract events provide additional traceability for recorded actions, but physical accuracy still requires dependable operational processes.

Repository: https://github.com/Sai-Narayana/plastic_trace

Drive Link : https://drive.google.com/drive/folders/1RVs-ySgKrBEDzkLp9



GMcXE9mOZfN8Haa?usp=sharing